

Lab 7: Infrastructure as Code, Including the Destroy

Declare infrastructure, apply it, prove it is idempotent, break it by hand and watch the tool notice, then remove it completely.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

One 2-hour block

MARKS

15

SUBMIT

icclab7-<ROLLNO>.pdf

OFFERING

Odd Semester 2026-27

Setup

Install OpenTofu (or Terraform; everything here applies to both).

```
tofu version
```

You may use your cloud free tier, or a local provider such as `docker` or `null` if you would rather not spend credit. The lessons are identical.

Part 1: Declare, plan, apply (4 marks)

1.1 Write a configuration creating at least three related resources, for example a network, a firewall rule and an instance, or a Docker network, volume and container.

1.2 Run `tofu init`, then `tofu plan`. **Read the plan.** Report how many resources it will add, change and destroy.

1.3 `tofu apply`. Verify the resources exist by an independent route: the CLI of the provider, or `docker ps`, not the tool that created them.

PITFALL · READ EVERY PLAN, EVERY TIME

`plan` is the only thing standing between you and an irreversible change. The line you are looking for is any resource marked **destroy** or **replace**. The first time you see a database marked for replacement will teach you this permanently; better that it is in a lab.

Part 2: Idempotence (4 marks)

- 2.1 Run `tofu plan` again immediately after applying. It must report **no changes**. Paste the output.
- 2.2 If it does report changes, you have found a **convergence bug**: some attribute you declared is normalised by the provider into a different form, so every apply sees a difference and the deployment never settles. Investigate and report what it was.
- 2.3 Explain, in three sentences, why a configuration that never converges is dangerous even though each individual apply appears to succeed.
- 2.4 Now change something **by hand**, outside the tool: rename a resource, change a firewall rule, stop a container. Run `tofu plan` again. Show that the tool detects the drift and states exactly what it will do about it.

Part 3: State (3 marks)

3.1 Look at the state file. Do **not** commit it. Explain in two sentences what it contains and why it exists.

3.2 Explain what would happen if two people applied the same configuration against two separate local state files. Be specific about the failure.

3.3 Configure a remote backend with locking, or, if that is not available to you, describe precisely how you would and what problem it solves.

3.4 Find one secret-shaped value in your state file, or explain why there is none. State the implication for how state must be stored.

Part 4: Environments are variables (2 marks)

4.1 Parameterise your configuration with variables so that `dev` and `prod` differ only by a values file, not by duplicated code.

4.2 Show `plan` output for both. Explain in two sentences why duplicating the configuration per environment guarantees they will drift apart.

Part 5: Destroy (2 marks)

5.1 `tofu destroy`. Paste the output.

5.2 Verify independently that everything is gone.

5.3 Some resources have a destroy **order** dependency: a certificate bound to a domain, a network with something still attached. Describe one such dependency in your configuration and what would happen if the tool got the order wrong.

What to submit

`icclab7-<ROLLN0>.pdf` : your configuration as text, the plan output at each stage, the no-changes proof from 2.1, the drift detection from 2.4, both environment plans, and the destroy output with independent verification.

MARKS	FOR
4	Part 1: a real multi-resource configuration, applied and verified
4	Part 2: idempotence proved and drift detection demonstrated
3	Part 3: a correct account of state, locking and secrets
2	Part 4: environments as variables
2	Part 5: complete destroy, verified, with the ordering discussion